

Autonomous Traffic Intersection Management System

Nader Yahia
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nader.yahia@stud.hshl.de

Md Mostafizur Rahman
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
md-mostafizur.rahman@stud.hshl.de

Ikramul Hassan Sazid
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
ikramul-hassan.sazid@stud.hshl.de

Muhammad Umer Hayat
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
muhammad-umer.hayat@stud.hshl.de

Abstract—Conventional traffic-light-controlled intersections rely on static or semi-adaptive timing strategies that often result in unnecessary delays, inefficient utilization of road infrastructure, and limited scalability under dynamic traffic conditions. With the emergence of autonomous and connected vehicles, reservation-based intersection management has become a promising alternative capable of improving both traffic efficiency and safety. This paper presents the design of an intelligent intersection management system based on centralized reservation scheduling for a four-way urban intersection. The proposed architecture incorporates conflict-aware vehicle coordination, dynamic reservation allocation, and real-time scheduling to enable collision-free intersection traversal while reducing vehicle waiting times. The system is developed as a cyber-physical architecture suitable for embedded implementation and future deployment on FreeRTOS-based platforms. Furthermore, the design is structured to support formal verification using UPPAAL, providing a foundation for validating safety, liveness, and timing properties before hardware implementation.

I. INTRODUCTION

The rapid development of autonomous vehicles, connected transportation systems, and Vehicle-to-Everything (V2X) communication is transforming the design of intelligent transportation infrastructures [1]. Unlike conventional vehicles that rely on traffic lights and driver decisions, connected autonomous vehicles can exchange information with infrastructure and other vehicles in real time, enabling coordinated and more efficient traffic management. This evolution has motivated the development of intelligent intersection management systems capable of replacing traditional traffic signals with dynamic reservation-based control strategies.

A. Motivation

Urban intersections are major contributors to transportation inefficiencies, with studies from the Federal Highway Administration (FHWA) highlighting their significant impact on

urban traffic congestion [2]. Conventional fixed-time traffic lights are designed for predictable traffic patterns and often perform inefficiently under dynamic and heterogeneous traffic conditions, resulting in increased waiting times and reduced intersection efficiency [3].

The advancement of autonomous vehicles, Vehicle-to-Everything (V2X) communication, and embedded computing enables a transition toward reservation-based intersection management, where vehicles coordinate their trajectories through intelligent controllers instead of relying on traditional traffic signals. Such approaches have demonstrated the potential to reduce travel delays, improve traffic flow, and enhance intersection utilization [4]. Furthermore, optimized vehicle coordination can reduce unnecessary acceleration, braking, and idle periods, contributing to lower fuel consumption and improved energy efficiency [5]. By reducing reliance on human decision-making in complex traffic scenarios, autonomous intersection systems can also mitigate human-factor-related driving errors and improve road safety [6].

This project investigates the integration of cyber-physical system principles, real-time scheduling algorithms, and embedded-system design techniques to develop a safe, efficient, and verifiable smart intersection controller. The proposed system focuses on conflict-aware vehicle coordination, dynamic reservation scheduling, and formal verification while preparing the architecture for future real-time embedded implementation.

B. Objectives

This project aims to investigate reservation-based intersection management as a potential solution for improving the efficiency and safety of future autonomous and connected transportation systems. The objective is to design and model an intelligent intersection controller capable of dynamically

coordinating vehicle movements while satisfying real-time and safety constraints.

The specific objectives are:

- Develop a formal traffic model for a four-way urban intersection with two lanes per approach.
- Design a conflict-aware vehicle coordination mechanism based on vehicle trajectory information.
- Develop a reservation-based scheduling strategy for dynamically assigning safe intersection access.
- Reduce vehicle waiting times and improve intersection throughput compared with conventional traffic control approaches.
- Ensure collision-free operation through conflict detection and arbitration mechanisms.
- Model the intersection controller as a cyber-physical system suitable for real-time execution.
- Prepare the system architecture for future embedded implementation using FreeRTOS.
- Establish a foundation for formal verification of safety and timing properties using UPPAAL.

The remainder of this paper is organized as follows. Section II defines the functional and non-functional requirements of the proposed system. Section III presents the main system use cases and operational scenarios. Section IV describes the intersection trajectory model, including the mapping of vehicle maneuvers to spatial quadrants and the identification of path conflicts. Section V presents the overall system architecture and its functional modules. Section VI introduces the UPPAAL model, the conflict-aware scheduling policy, the quadrant-based reservation controller, and the formal verification of the proposed system. Finally, Section VII describes the hardware/software partitioning and its realization, including the FreeRTOS-based ARM software and the FPGA-based VHDL hardware implementation. Section VIII concludes the paper by summarizing the main results, limitations, and future enhancements.

II. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

The proposed autonomous intersection controller is designed according to a set of functional and non functional requirements. These requirements define the expected system capabilities, safety constraints, and real-time limitations. The main design requirements considered in this work are summarized as follows:

- **Dynamic vehicle coordination:** The system shall acquire vehicle state information and generate safe crossing schedules based on vehicle intentions and trajectories (FR-1, FR-3, FR-5).
- **Conflict-free intersection access:** The controller shall detect spatial-temporal conflicts and prevent multiple vehicles from occupying incompatible conflict regions simultaneously (FR-4, NFR-9).
- **Real-time scheduling:** The scheduling mechanism shall update vehicle reservations within bounded execution time while maintaining deterministic behavior (NFR-1, NFR-2).

- **Safety under abnormal conditions:** The system shall support emergency priority handling and transition to safe states during communication or sensor failures (FR-10, NFR-5, NFR-14).
- **Embedded feasibility:** The proposed architecture shall consider computational limitations and real-time execution requirements for future embedded implementation (NFR-8).

Together, these requirements establish the design basis for the proposed autonomous intersection controller. They define the required vehicle coordination, conflict-free reservation, real-time scheduling, safety behavior, and embedded implementation constraints. The subsequent system architecture and formal UPPAAL model in Section V are developed to satisfy these requirements.

III. USE CASE MODEL

Figure 1 defines *Cross Intersection Safely* (UC1) as the central system function. An approaching vehicle initiates UC1, while UC1 always includes *Validate Vehicle State & Intent* (UC2). The V2X and roadside infrastructure support UC2 by supplying the vehicle's position, speed, direction, and intended maneuver.

Three conditional use cases extend UC1. *Resolve Conflicting Path Request* (UC4) is activated in scenario S3 when trajectories compete for the same intersection resources. *Grant Emergency Priority Passage* (UC3) is activated in S4 for an emergency vehicle, which is treated as a specialized approaching vehicle. *Operate in Degraded/Safe Mode* (UC5) is activated in S5 when sensor or communication data are missing, delayed, or unreliable. Traffic operators or maintenance personnel supervise this mode. Therefore, every crossing request follows the normal validation path, while conflict, emergency, and failure handling are applied only when their corresponding conditions occur.

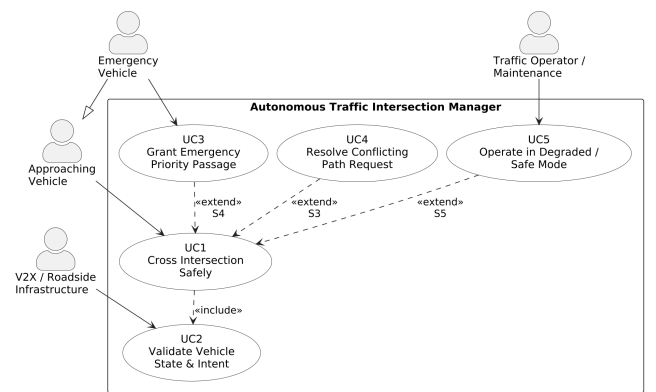


Fig. 1. Use-case diagram of the autonomous traffic intersection manager.

IV. RESERVATION AND CONFLICT CONTROL

The intersection is divided into four spatial quadrants, Q1–Q4, and the twelve standard maneuvers M1–M12 are mapped to the ordered quadrant sequence occupied by each vehicle. A request conflicts with an existing reservation if the two

vehicles have at least one common quadrant at overlapping time intervals. controller decision chain is as follows:

Non-conflicting vehicles may be assigned the same or overlapping time slots, increasing intersection throughput. Conflicting moves are separated by arbitration and separate reservations. order of the crossing of the normals is first-come, first-served (FCFS). Emergency priority may change the order of this but never skips vehicle validation or checking for collision. If a safe reservation is not possible, the controller instructs the vehicle to wait or stop.

Request → Validate → Conflict Check → Grant / Wait / Stop

V. OPERATIONAL SCENARIOS

The required behavior is described in Table I. S1 and S2 cover only the base use case; S3–S5 are exactly the guards of the three extending use cases, therefore each scenario points to an element of Fig. 1 and each extension to a required behavior. S2 is the disjoint path case of Table II.

Scenario	Description
S1	Normal Operation: Validate the vehicle request, apply the scheduling policy, perform conflict detection, reserve the required quadrants, and grant crossing permission.
S2	Non-Conflicting Requests: Allow compatible vehicle movements to proceed simultaneously by assigning safe overlapping reservation intervals.
S3	Conflicting Requests: Resolve competing requests through arbitration and assign non-overlapping reservation time slots.
S4	Emergency Operation: Suspend conflicting traffic and safely prioritize emergency vehicle passage.
S5	Failure Handling: Treat unavailable sensor or communication data as uncertain and transition affected vehicles to a safe wait or stop state.

TABLE I
OPERATIONAL SCENARIOS

VI. INTERSECTION TRAJECTORY ANALYSIS

Trajectory mapping discretizes the twelve directional maneuvers M1–M12 into standardized straight and turning paths. Spatial resource allocation links each maneuver to an ordered sequence of Q1–Q4, allowing the controller to reserve only the parts of the intersection required by a vehicle. This representation converts a continuous vehicle path into a compact time-space reservation that can be checked efficiently.

Conflict identification compares the quadrant-time occupancy of All active reservations. For example, M1, M9 and M10 have the same Q2 → Q3 trajectory and thus constitutes a scheduling bottleneck. Their reservations must be separated in time. In contrast, vehicles with non-overlapping requested quadrant-time intervals may cross at the same time. Trajectory model helps to avoid collisions, to guarantee safe parallel movement and to identify heavily demanded intersection resources at an early stage.

Quadrant path	Assigned maneuvers
Q2 → Q3	M1 (N→S), M9 (N→W), M10 (W→S)
Q4 → Q1	M2 (S→N), M11 (S→E), M12 (E→N)
Q4 → Q3	M3 (E→W), M6 (E→S), M7 (S→W)
Q2 → Q1	M4 (W→E), M5 (N→E), M8 (W→N)

TABLE II
TRAJECTORY GROUPS

VII. SYSTEM ARCHITECTURE

A. System Architecture Overview

The proposed autonomous intersection controller follows a modular hierarchical architecture, as shown in Fig. 2. The system is divided into four main layers: perception, decision, execution, and state management. The perception layer, consisting of the Sensor Subsystem and Lane State Builder, converts vehicle and environmental data into a structured traffic representation. The decision layer performs conflict analysis and scheduling through the Conflict Detection, Arbitration, Scheduling, and Verification modules. The execution layer, including the Reservation Manager and Vehicle Interface, applies the generated decisions and manages vehicle authorization. The Global State Manager maintains a consistent system state shared across all modules.

This modular decomposition improves requirement traceability, scalability, and verification by separating safety-critical functions into well-defined components [7]. It also provides a suitable foundation for the subsequent formal modeling and hardware/software implementation.

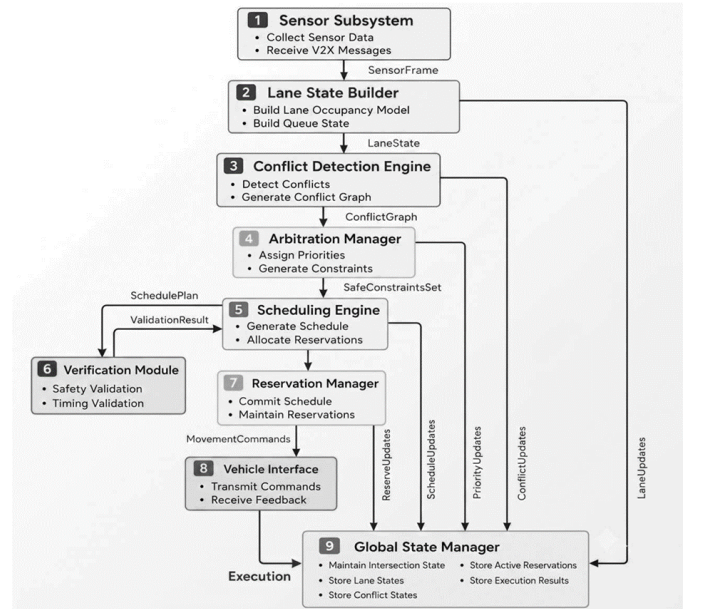


Fig. 2. High-level architecture of the autonomous intersection controller.

B. System Operational Workflow

The operational workflow of the autonomous intersection controller, illustrated in Fig. 3, describes the interaction between the vehicle interface and the Intersection Coordination Unit. The process starts when a vehicle approaches the intersection and transmits its state information and movement intention through the V2X interface. The received information is processed by the Lane State Builder to update the global intersection state and generate a vehicle request. The Scheduler evaluates pending requests based on the defined scheduling policy, considering waiting time and movement-dependent cost. The selected request is then passed to the Conflict Detection and Reservation modules, where the required intersection resources are evaluated. The controller checks the availability of the required spatial quadrants and assigns a time-space reservation only if no conflicting reservation exists.

If the required resources are unavailable, the request remains in the waiting queue until a valid reservation opportunity becomes available. Once the reservation is granted, the Intersection Controller authorizes the vehicle movement, executes the reserved crossing interval, and releases the occupied quadrants after completion. Emergency handling is represented as an additional control path through the Emergency Handler module. The current implementation provides the structural mechanism for emergency-state integration, while advanced emergency preemption and priority scheduling are considered future extensions.

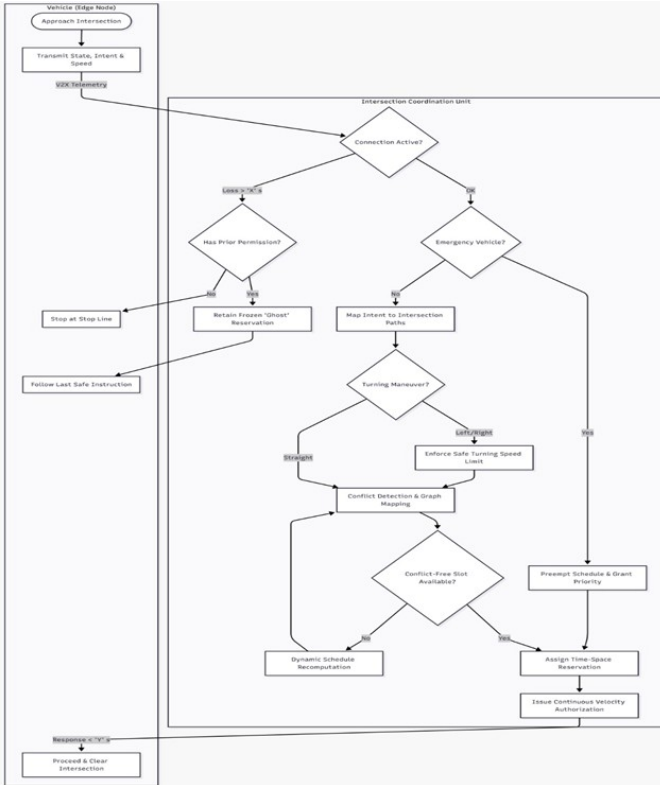


Fig. 3. Operational workflow of the autonomous intersection controller.

C. System Interaction Sequence

The sequence diagram in Fig. 4 illustrates the interaction between the main components of the autonomous intersection controller during a vehicle request cycle. The process begins when vehicle state information and movement intent are received through the V2X interface. The Lane State Builder updates the global intersection state, which is then evaluated by the Conflict Detection and Scheduling modules.

Based on the scheduling decision, the Reservation Manager evaluates the availability of the required spatial resources and creates a valid time-space reservation. After successful reservation, the Intersection Controller grants movement authorization to the vehicle and updates the global system state after execution completion. This sequence demonstrates the interaction between perception, decision-making, reservation, and execution layers.

Emergency handling is represented as an additional interaction path through the Emergency Handler. In the current implementation, this mechanism provides the structural extension point for future emergency-priority scheduling and preemption strategies.

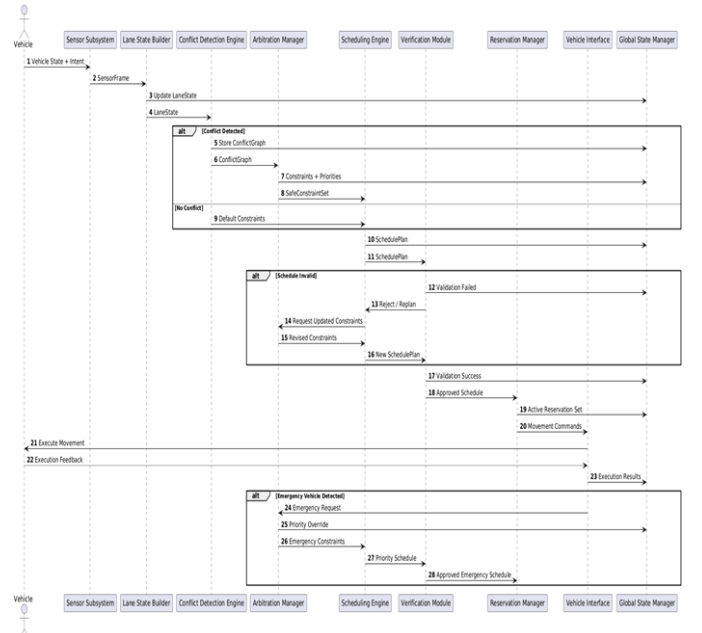


Fig. 4. Interaction sequence of the autonomous intersection controller.

VIII. UPPAAL MODELING AND FORMAL VERIFICATION

The proposed autonomous intersection controller was formally modeled and verified using UPPAAL timed automata [8]. The model represents the interaction between vehicle generation, request management, scheduling, spatial reservation, movement execution, and emergency handling. The intersection is abstracted as a set of four mutually exclusive spatial resources, following the concept of resource reservation used in autonomous intersection management approaches [4],

denoted by Q_1 – Q_4 , which are reserved according to the spatial requirements of each vehicle movement.

A. Formal Intersection Model

The UPPAAL model consists of several interacting timed automata that represent the main functional components of the proposed controller. Each automaton has a specific responsibility within the overall control process. The main templates are summarized in Table VIII-A.

Template	Function
<i>VehicleGenerator</i>	Generates vehicle requests and assigns movement types.
<i>Vehicle</i>	Represents the lifecycle of an individual vehicle request.
<i>Scheduler</i>	Selects the next vehicle using waiting time and movement cost.
<i>IntersectionController</i>	Manages quadrant reservation, movement execution, and resource release.
<i>EmergencyHandler</i>	Handles emergency detection and priority-state transitions.

TABLE III
UPPAAL TIMED AUTOMATA AND THEIR FUNCTIONS

The system is bounded to four simultaneously modeled vehicle instances, denoted by V_0 – V_3 . Each vehicle is assigned a movement request and remains in the system until its movement has been completed and its associated resources have been released.

B. Movement and Quadrant Resource Representation

The intersection is represented using four spatial resources, Q_1 – Q_4 . Each vehicle movement is mapped to the quadrants that it occupies during intersection traversal. The twelve supported movements are grouped according to their required spatial resources, as shown in Table VIII-B.

Movement Group	Required Quadrants	Movements
G1	Q_2, Q_3	M1, M9, M10
G2	Q_4, Q_1	M2, M11, M12
G3	Q_4, Q_3	M3, M6, M7
G4	Q_2, Q_1	M4, M5, M8

TABLE IV
MOVEMENT-TO-QUADRANT RESOURCE MAPPING

Each quadrant is modeled as a Boolean exclusive resource. A value of `false` indicates that the corresponding quadrant is available, whereas `true` indicates that it is currently reserved. Consequently, a vehicle request can only be accepted if all quadrants required by its movement are available. For a movement requiring quadrants Q_a and Q_b , a reservation is therefore permitted only if

$$Q_a = \text{false} \wedge Q_b = \text{false}. \quad (1)$$

Once the reservation is accepted, both resources are marked as occupied until the vehicle completes its assigned execution

interval. This mechanism provides mutual exclusion between movements that share spatial resources.

C. Vehicle Lifecycle and Request Management

Each vehicle is modeled using a finite-state lifecycle:

$$\text{Idle} \rightarrow \text{Request} \rightarrow \text{Waiting} \rightarrow \text{Done}. \quad (2)$$

An inactive vehicle remains in the *Idle* state. After activation, a movement request is generated and the vehicle transitions to the *Request* state. The request is then registered in the system and the vehicle enters the *Waiting* state until it is selected by the scheduler and successfully admitted to the intersection. Following the completion of the execution interval, the vehicle transitions to the *Done* state. The associated request and vehicle state are then reset, allowing the vehicle instance to be reused for a subsequent request. The number of waiting vehicles is bounded by

$$0 \leq L_q \leq L_{\max}, \quad (3)$$

where L_q denotes the current queue length and $L_{\max} = 4$ represents the maximum number of modeled waiting vehicles.

D. Conflict-Aware Scheduling Policy

The scheduler combines waiting-time aging with a movement-dependent cost. For each pending vehicle request i , the scheduler maintains a waiting-time value and an associated movement cost. The priority score is defined as

$$S_i = W_i - C_i, \quad (4)$$

where W_i represents the accumulated waiting time of request i and C_i denotes the movement-dependent cost. The waiting time is increased during scheduling cycles, allowing requests that remain in the queue to gradually gain priority. In the implemented model, straight movements are assigned a lower cost, whereas turning movements are assigned a higher cost. The corresponding cost classification is

$$C_i = \begin{cases} 1, & \text{straight movement,} \\ 2, & \text{turning movement.} \end{cases} \quad (5)$$

The scheduler selects the highest-scoring valid request for subsequent reservation evaluation. This mechanism combines aging-based fairness with movement-dependent scheduling preferences. Importantly, the score determines the scheduling order, whereas the actual spatial conflict prevention is enforced by the quadrant reservation mechanism described in Section VIII-E.

E. Quadrant-Based Reservation Controller

The *IntersectionController* implements the reservation-based resource management mechanism. Its operation is represented by the following state sequence:

$$\text{Free} \rightarrow \text{Reserve} \rightarrow \text{Execute} \rightarrow \text{Release}. \quad (6)$$

A selected vehicle can transition from the *Free* state to the *Reserve* state only if its required quadrants are available. During reservation, the required spatial resources are marked as occupied. The vehicle can then enter the *Execute* state and traverse the intersection during the assigned execution interval. The execution duration is bounded by the configured slot duration:

$$T_{\text{slot}} = 5. \quad (7)$$

After the execution interval has elapsed, the controller transitions to the *Release* state. The reserved quadrants are cleared, the vehicle request is removed from the pending request set, and the vehicle instance is deactivated. The controller then returns to the *Free* state. This state sequence ensures that spatial resources are reserved before execution and released only after the assigned movement has completed.

F. Emergency Handling

The UPPAAL model includes a dedicated *EmergencyHandler* automaton to reserve an explicit control path for future emergency-priority extensions. Its state structure is represented as

$$\text{Normal} \rightarrow \text{Emergency} \rightarrow \text{Normal}. \quad (8)$$

The automaton provides a dedicated emergency state that can be activated when an emergency condition is detected and subsequently cleared through a priority-override transition. In the current implementation, this structure serves as a placeholder for future emergency-handling functionality and does not yet implement complete emergency vehicle preemption or priority-based reservation. The integration of advanced emergency handling, including safe preemption and priority reservation, is therefore considered part of the future work described in Section X.

G. Formal Verification

The UPPAAL verifier was used to evaluate the safety, timing, and operational properties of the proposed controller. The verification objectives were derived from the functional and non-functional requirements defined in Section II. The main verification properties are summarized in Table VIII-G.

The principal safety property is the prevention of invalid reservations. This is expressed by ensuring that the controller cannot enter the reservation state without a valid selected vehicle:

$$A \models \neg (\text{Reserve} \wedge \text{selectedVehicle} = -1). \quad (9)$$

The scheduling deadline is verified using the scheduling clock and the configured deadline:

$$A \models \neg (\text{Compute} \wedge \text{sched_clk} > T_{\text{sched}}), \quad (10)$$

where T_{sched} represents the maximum allowed scheduling computation time.

Property	Verification Objective
Valid reservation	A reservation cannot be entered without a valid selected vehicle.
Scheduling deadline	The scheduling computation remains within the defined deadline.
Operational reachability	The scheduler can reach its computation state.
Execution reachability	A selected vehicle can eventually reach the execution state.
Deadlock freedom	The system does not reach a state in which no further transition is possible.
Quadrant mutual exclusion	Conflicting movements cannot simultaneously reserve shared quadrants.
Eventual service	Pending requests can eventually complete under the modeled operating conditions.

TABLE V
FORMAL VERIFICATION PROPERTIES

In addition to safety properties, reachability queries are used to verify that the principal operating states of the system can be reached. This includes the scheduler computation state and the vehicle execution state. The verification results confirm that the modeled controller satisfies the defined safety and timing properties under the modeled operating conditions. In particular, the reservation mechanism prevents invalid resource allocation, the scheduler operates within its defined timing constraint, and the system can progress through the intended request, reservation, execution, and release sequence.

The UPPAAL model therefore provides an executable formal representation of the proposed reservation-based autonomous intersection controller. By combining bounded vehicle request management, aging-based scheduling, quadrant-level resource reservation, timed execution, and emergency-state handling, the model captures the principal operational behavior of the proposed system. The successful verification of the defined safety, timing, and operational properties provides formal evidence that the modeled controller satisfies its principal system requirements.

IX. HARDWARE-SOFTWARE PARTITIONING AND REALIZATION

The proposed autonomous intersection controller follows a hardware–software co-design approach in which computational tasks are divided between an ARM processor running FreeRTOS and an FPGA implementing the time-critical arbitration logic. This partitioning combines the flexibility of software with the deterministic execution of dedicated hardware, allowing the controller to satisfy both functional and real-time requirements.

A. Hardware–Software Partitioning

The ARM processor performs the high-level control functions, including vehicle request management, scheduling, communication, and overall system coordination. In contrast, the FPGA executes the localized traffic arbitration functions that

require predictable timing. These include movement decoding, conflict detection, quadrant reservation, and resource release.

ID	Functional Block	HW / SW
F1	Vehicle Detection	Hardware
F2	Movement Identification	Software
F3	Queue Management	Software
F4	Scheduler	Software
F5	Quadrant Manager	Software
F6	Reservation Manager	Software
F7	Intersection Controller	Hybrid
F8	Emergency Handler	Hybrid
F9	Communication Manager	Hybrid
F10	Timer & System Clock	Hardware
F11	Diagnostics & Logging	Software

TABLE VI
HARDWARE/SOFTWARE PARTITIONING OF FUNCTIONAL BLOCKS

Justification for Hardware/Software Partitioning

The partitioning of these functional blocks into hardware, software, or a hybrid approach is driven by their specific computational, physical, and flexibility requirements. Pure hardware implementations are utilized where direct physical interfacing and precise timing are strictly required. For instance, Vehicle Detection (F1) relies entirely on hardware because physical sensors generate electrical signals that must be acquired naturally through GPIOs, ADCs, or specific camera and radar interfaces. Similarly, the Timer and System Clock (F10) requires the high-precision timing and hardware interrupts provided natively by the MCU, leaving software to merely handle their configuration.

Conversely, functions requiring flexibility, complex decision-making, and dynamic data handling are allocated to software. Movement Identification (F2) is implemented in software to allow its logic to easily evolve as it processes incoming sensor and V2X information. Software is also the natural choice for managing dynamic data structures, which is why Queue Management (F3) and the Quadrant Manager (F5) are software-based; the latter simply maintains intersection occupancy states using low-overhead variables or bitmasks. Higher-level decision logic, including the Scheduler (F4) for handling fairness policies, emergency handling, and optimization, as well as the Reservation Manager (F6) for atomic quadrant availability checks, are kept in software due to their close coupling and the fact that decision logic is much easier to verify and update in code. Furthermore, Diagnostics and Logging (F11) are inherently software responsibilities, naturally handling tasks like statistics gathering, debugging, and event storage.

The communication between both domains is performed through an AXI-Lite interface. After selecting the next vehicle request, the software transmits the movement identifier together with a start signal to the FPGA. The hardware performs the arbitration procedure and returns the corresponding

grant, conflict, reservation state, and completion signals to the software.

B. FreeRTOS Execution and Technical Operation

The software layer uses the FreeRTOS real-time operating system, which separates system tasks according to their priority level. A VehicleInputTask handles V2X and sensing information, verifying the payload and enforcing a 75% AI confidence threshold. The SchedulerTask implements a First-Come First-Served scheduler policy, which manipulates the hardware through memory-mapped AXI-Lite registers.

Lastly, an EmergencyTask runs at the highest priority level and utilizes an override flag, allowing it to preempt any other running task. FreeRTOS functions as a preemptive priority-based Real-Time Operating System (RTOS) on the ARM processor, acting as the master supervisor. It manages CPU allocation and task scheduling according to the established policies while also handling data exchange between the software application and the underlying FPGA hardware layer. The FreeRTOS scheduler manages the CPU by prioritizing tasks, allocating more processing time to higher-priority ones according to their defined Task Control Blocks (TCBs). The priority hierarchy establishes the EmergencyTask at the highest priority, followed by the SchedulerTask, V2XReceiverTask, FPGAInterfaceTask, and MonitorTask. If a high-priority event occurs, such as a dedicated sensor signal being processed, FreeRTOS triggers a context switch, saving the active task's state and instantly executing the emergency override to halt normal reservations.

As a result, when an emergency event is detected and the emergency sensor is processed, FreeRTOS context-switches to the highest priority task, thus allowing the emergency override to interrupt normal operations and proceed to a task-specific emergency protocol.

With multiple threads executing concurrently and sharing the same memory space, FreeRTOS implements several mechanisms to avoid race-conditions and ensure memory safety. The mutual exclusion locks (mutexes) are used to synchronize threads accessing shared resources such as the critical section code for the Q1-Q4 state (`quadrant_mutex`) and the directional vehicle queues (`queue_mutex`). In addition, the deadlock problem is avoided by enforcing an ordering of mutex locks such as first acquiring the emergency mutex followed by the queue and then the quadrant mutex. Secondly, no I/O blocking operations are performed when a mutex is acquired. Similarly, binary semaphores are employed to synchronize tasks when processing certain events such as when the input task advertises a vehicle arrival and consumes the `vehicle_available_sem` before releasing the CPU back to the scheduler. This approach prevents the SchedulerTask from unnecessarily wasting power polling an empty vehicle queue.

Finally, the FreeRTOS communication mechanism is used to interface with the custom FPGA hardware accelerators through memory-mapped AXI4-Lite registers. Pointers are casted to volatile types to ensure that the compiler makes no

assumptions about the accessed hardware addresses. The task scheduler makes use of the `vTaskDelay()` function which in turn places the task in the blocked state for a specified number of RTOS ticks. This allows the time-critical tasks to consume the CPU while the lower-priority tasks wait for the hardware operations such as the FPGA slot timer or the UART FIFO bits to be cleared before they yield back the CPU.

A few examples of tasks which require waiting for hardware are the interface task which polls the status register until the FPGA slot timer is cleared and the input task which waits for the UART FIFO bits to be available for transmitting information.

C. Timing Analysis and Verification

Strict timing parameters guarantee real-time safety. A worst-case response time (WCRT) model bounds the maximum execution delay. Assuming N conflicting vehicles ahead, the WCRT is expressed as:

$$\text{WCRT} = \text{SENSOR_PERIOD} + \text{SCHED_DEADLINE} + (N \times \text{SLOT_TIME}) + \text{SLOT_TIME}$$

This formulation decouples spatial safety from operational latency. Hardware correctness is validated through a unified VHDL testbench. It rigorously checks deterministic responses under valid, conflicting, and invalid intersection scenarios.

D. FPGA Arbitration Module

In the scope of the project, we have used System-on-Chip (SoC) Field-Programmable Gate Array (FPGA). It is based on the Xilinx Zynq-7000 or Intel Cyclone V devices. This choice was made for the projects needs for a deterministic hardware execution and a flexible software-defined design, which are compatible with each other. An ARM processing system has been used which is connected to the custom FPGA intersection accelerator through AXI4-Lite high-speed interconnect buses.

The hardware acts as a spatial-temporal accelerator. It contains five core modules.

- **Movement Decoder:** Converts the received movement identifier into the required intersection quadrants.
- **Conflict Detection:** Checks whether the requested quadrants are currently occupied.
- **Reservation Arbiter:** Grants or rejects the request according to the conflict status.
- **Quadrant Manager:** Maintains the reservation status of the four intersection quadrants.
- **Slot Timer:** Releases the reserved quadrants after the assigned crossing interval.

This hardware implementation provides deterministic execution of the reservation mechanism while reducing the computational load on the software scheduler.

E. RTL Verification

The VHDL design was verified using a behavioral simulation in Vivado. A dedicated test bench generated representative traffic scenarios, including valid requests, conflicting requests,

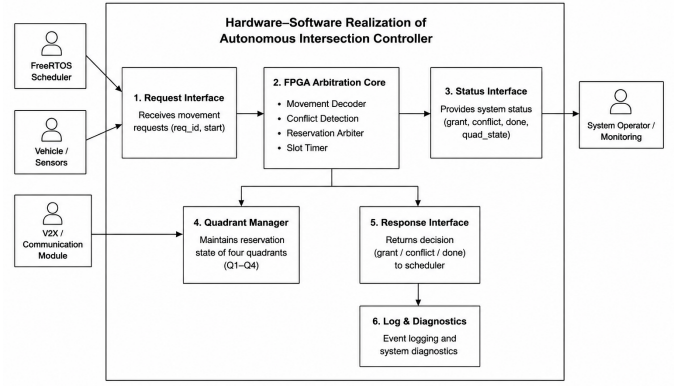


Fig. 5. Hardware-software realization architecture.

```
process(clk, reset)
begin
    if reset = '1' then
        grant <= '0';
        reserve <= '0';
    end if;

    elsif rising_edge(clk) then
        if start = '1' then
            if conflict_int = '0' then
                grant <= '1';
                reserve <= '1';
            else
                grant <= '0';
                reserve <= '0';
            end if;
        else
            grant <= '0';
            reserve <= '0';
        end if;
    end if;
end if;
```

Fig. 6. Reservation Arbiter

sequential vehicle execution, slot-time release, and invalid request handling.

The simulation waveform confirms that incoming movement requests are correctly decoded, conflicts are detected before reservation, and only safe requests receive a grant signal. The quadrant reservation state is updated throughout the execution process, and the hardware generates a completion signal after the assigned execution interval. These results demonstrate the correct operation of the hardware arbitration logic before FPGA synthesis.

The hardware realization demonstrates that the reservation

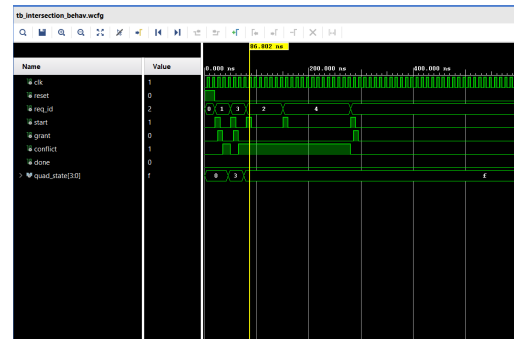


Fig. 7. Testbench Waveform

and conflict-detection algorithm developed and formally verified in UPPAAL can be translated into synthesized RTL suitable for FPGA implementation. Although the current implementation represents a simplified prototype, it preserves the fundamental reservation-based coordination strategy and establishes a practical interface between the software scheduler and the dedicated hardware. This realization provides an important step toward future deployment of real-time autonomous intersection management systems.

X. CONCLUSION

This paper presented a reservation-based autonomous intersection control architecture designed to address the limitations of conventional traffic-light-based intersection management in future intelligent transportation systems. The proposed approach replaces fixed-phase intersection control with vehicle-level coordination based on vehicle state information, movement intentions, conflict detection, and time-space reservation of shared intersection resources. The system requirements, operational scenarios, and intersection movement model were defined to establish the functional and safety basis of the proposed controller.

A formal UPPAAL model was developed to represent the interaction between vehicle requests, scheduling, quadrant-based resource reservation, movement execution, and resource release. The proposed controller combines waiting-time-based scheduling with movement-dependent cost and uses exclusive quadrant reservations to prevent incompatible vehicle movements from accessing shared spatial resources simultaneously. The formal verification results demonstrate that the modeled system satisfies the defined safety, timing, and operational properties under the considered operating conditions.

Furthermore, the project investigated a hardware/software co-design realization of the controller. The decision-making and system-management functions were assigned to an ARM processor running FreeRTOS, while deterministic and time-critical intersection functions were allocated to an FPGA-based VHDL implementation. The AXI-Lite communication interface provides a structured communication mechanism between the software and hardware domains. This partitioning demonstrates how the formally modeled intersection controller can be mapped to a heterogeneous embedded platform while preserving the separation between flexible software-based coordination and deterministic hardware-based safety enforcement. RTL simulation results for FPGA arbitration module prove that the feasible implementation of reservation logic on embedded hardware is possible, which is significant in terms of its real-time use.

Despite these results, the current implementation has several limitations. The traffic model uses a bounded number of vehicle requests and a simplified intersection abstraction, while the emergency-handling mechanism is currently represented as an extensible control structure rather than a complete priority and preemption mechanism. In addition, the hardware/software realization represents a prototype architecture and does not yet

address the full complexity of large-scale multi-intersection traffic networks or realistic urban traffic conditions.

Future work will focus on extending the scheduling strategy, improving the parallel processing capabilities of the FPGA implementation, and supporting more advanced communication mechanisms for multiple simultaneous requests. Adaptive and prediction-based scheduling methods could further improve the system under dynamic traffic conditions. In addition, AI-based traffic prediction and coordination between multiple FPGA-controlled intersections could extend the proposed architecture toward scalable intelligent transportation networks.

Overall, this project demonstrates a complete development methodology for a reservation-based autonomous intersection controller, combining system requirements engineering, formal timed-automata modeling, verification, and hardware/software co-design. The results establish a foundation for future real-time intersection management systems in which deterministic hardware-based resource control and flexible software-based decision-making are integrated to provide safe, scalable, and intelligent traffic coordination.

REFERENCES

- [1] J. M. Z. P. *et al.*, "Vehicle-to-everything (v2x) communication in connected and autonomous vehicles: A survey," *IEEE Communications Surveys & Tutorials*, 2020.
- [2] Federal Highway Administration, "Traffic congestion and reliability: Linking solutions to problems," 2020.
- [3] D. B. F. *et al.*, "Traffic signal control with connected and automated vehicles: A review," *IEEE Transactions on Intelligent Transportation Systems*, 2019.
- [4] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008.
- [5] M. Barth and K. Boriboonsomsin, "Energy and emissions impacts of a freeway-based dynamic eco-driving system," *Transportation Research Part D: Transport and Environment*, vol. 14, no. 6, pp. 400–410, 2009.
- [6] J. R. Treat, N. S. Tumbas, S. T. McDonald, *et al.*, "Tri-level study of the causes of traffic accidents," *National Highway Traffic Safety Administration*, 1979.
- [7] A. Kossiakoff, W. N. Sweet, S. J. Seymour, and S. M. Biemer, *Systems Engineering: Principles and Practice*. Wiley, 3rd ed., 2011.
- [8] UPPAAL Team, "Uppaal documentation and user guide." <https://docs.uppaal.org/>. Accessed: 2026-07-18.